

Project Report - ECE 176

Alexandra Dinh, Amrit Chahal

Abstract

We have implemented a fitness trainer model using the UCF-101 dataset to detect the difference between “good” and “bad” exercise form for four common bodyweight exercises (lunges, pullups, pushups, and squats). As part of our experimental study, we tested various models, including a feedforward neural network (MLP), a convolutional neural network (ResNet-18), a recurrent neural network (LSTM), XGBoost, and Transformer. We conducted a performance analysis of all models and selected the best model to implement a real-time webcam system that provides exercise form feedback. This report will cover key methods, experiments, results, and future directions.

1. Introduction

Bodyweight and at-home exercise are becoming increasingly more popular, and with that comes a growing need for exercise form correctors. Whether achieved with a personal trainer or an online application, it is crucial to have, as it prevents injury and ensures effective results. Time and financial constraints can limit access to getting this important feedback, which is why we are building a solution that is accessible and easy to integrate into daily routine.

One of the main challenges in this problem is defining a standard for good form that the model can recognize. With the diverse characteristics of the human body, the model must be able to accurately handle different body types. Additionally, the model should also account for external factors such as lighting, camera angle, and clothing.

Our problem involved processing videos from the UCF-101 dataset across the four categories mentioned in the abstract. The challenge we faced was that the videos didn’t have form labels, only the type of exercise. We designed a custom rule-based labelling pipeline, which we achieved by carrying out angle calculations on MediaPipe Pose keypoints extracted from video frames. We also accounted for camera orientation (subjects in videos could face either the camera or off to the side). This produced a result of each frame in each video having either a “good” or “bad” label, with a total of eight class labels.

We then trained and evaluated the performance of five models:

- Baseline MLP with 4 features (not included in final notebook)
- MLP with 16 features
- ResNet-18
- bi-directional LSTM
- XGBoost
- Transformer

2. Related Work

We used the MediaPipe Pose model as the foundation for our solution. A study by Shashank Negi, et al. (2023) involved training a model built on both the OpenPose and MediaPipe Pose frameworks to identify various yoga stances (e.g. Warrior pose, Tree-pose, T-pose) from a live camera feed and provide real-time feedback on the user's posture. The study noted that the model performed well under ideal conditions but produced inconsistent behaviour under unintended circumstances. Nevertheless, their system produced around 94% accuracy, which is why we decided to use MediaPipe Pose for our solution as well.

For our action recognition videos, we used the UCF-101 dataset, introduced by Soomro et al. (2012). This dataset consists of 13320 videos across 101 action categories, providing us with a diverse range of data for our model to test on. We used the provided train/test split from the UCF-101 website and filtered out the four classes to use for our model.

A paper by Chen and Guestrin (2016) motivated us to use a tree-boosting algorithm (XGBoost) as one of our testable models in our project. The paper cites XGBoost as a fast, scalable model that can output high accuracy even with limited resources. Due to limitations on the sheer size of our dataset, we were prompted to try XGBoost, using our calculated features.

3. Method

a. Data Loading and Preprocessing

We first created functions to extract frames and subsequently keypoints from the videos, saving the keypoints to two CSV files for the train and test splits. We then read the CSV files into a data frame for easier computation. We then removed unnecessary keypoints to increase efficiency and optimize file size.

b. Feature Computation and Labelling

We defined our functions to compute angles, including specific computations and constraints required for each exercise to receive either a “good” or “bad” form label. We then applied those labels to each data frame to obtain the respective class labels. To verify sufficient spread of data, we normalised the value counts to ensure an even distribution of each exercise across both the train and test splits. Finally, we implemented a visualisation function to display four “good” and four “bad” labels per exercise group to ensure the labels produced were sufficiently accurate.

c. Model Training

We first trained a baseline MLP with four features. It is not included in our final Jupyter Notebook, but the model achieved close to 45% accuracy. We then moved on to training five additional models, including adding more features to the MLP, noting how they performed across the different classes, using overall accuracy and F1 score (a metric that combines precision and recall) to compare the models’ performance with respect to each other.

d. Visualisation and Plots

We ended our experiments by displaying the four most important results for each model:

- Overall accuracy vs macro F1 (average F1 score across all classes equally)
- Accuracy progression line for all models
- Per-class F1 heatmap (displays which classes were best or worst learned by each model)
- Per-class F1 groups bar graph

4. Experiments

a. Dataset

Our primary dataset is UCF-101’s action recognition dataset, which contains 13320 videos. From that, we filtered out the four exercises we focus on in our project, yielding 441 videos. The dataset provides a dedicated train/test split that we used for our experiments. The videos in the dataset are stored as AVI files; they were filmed with a diverse range of lighting, camera angles, and subject appearances, all of which make our model more robust.

To supplement the UCF-101 dataset, we also scraped exercise tutorial videos from YouTube using yt-dlp, downloading up to 8 videos per exercise using “side-view” and “front-view” queries. All our videos were processed and extracted using the same pipeline.

Our data format for model training was a data frame with one row per frame. For MLP, we used 16 feature columns from our angle calculations. For sequence models, the frames were grouped into sliding windows of 20 frames with a stride of 5.

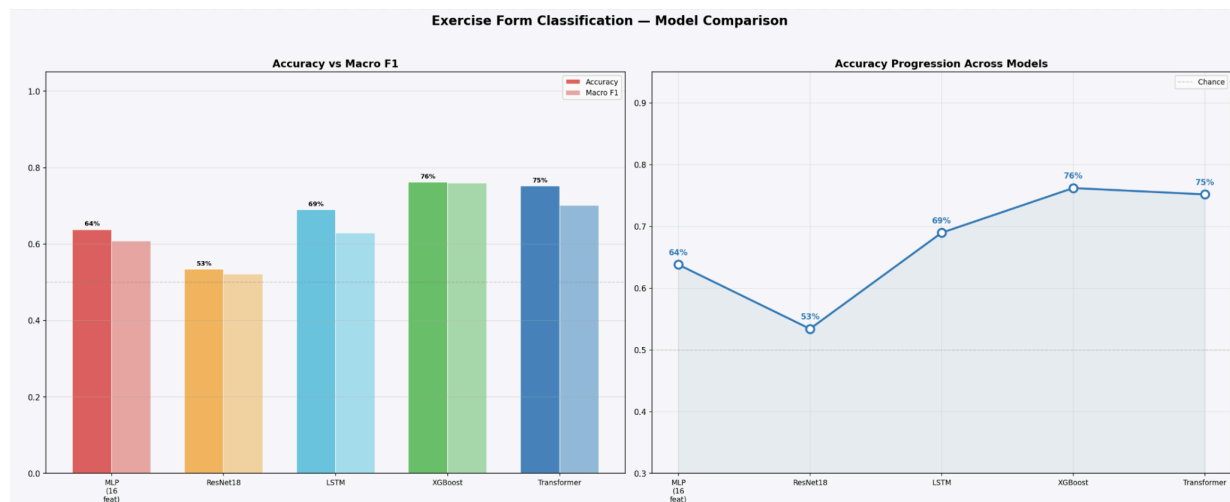
b. Results

We trained and evaluated on five models with varying degrees of accuracy. All models were evaluated on the same test set, and performance was measured by accuracy, macro-F1 score, and per-class F1 score. Below are the results for the models when trained with UCF-101 alone and UCF-101 + YouTube.

UCF-101 ALONE

Model	Accuracy (%)	Macro-F1 Score
Baseline MLP (4 feats)	45	0.28
MLP (16 feats)	64	0.61
ResNet-18	53	0.52
LSTM	69	0.63
XGBoost	76	0.76
Transformer	75	0.70

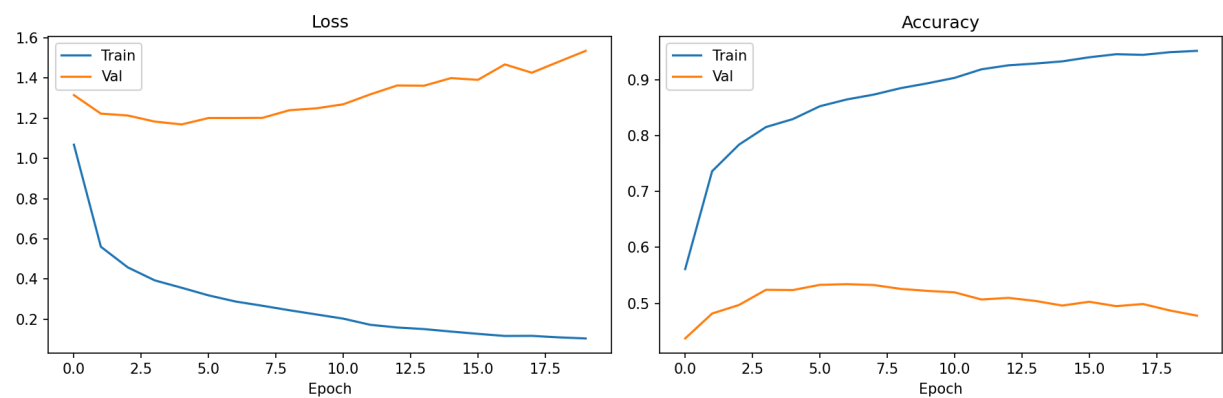
(Left) Accuracy vs Macro F1 Score, (Right) Accuracy Progression Across all Models



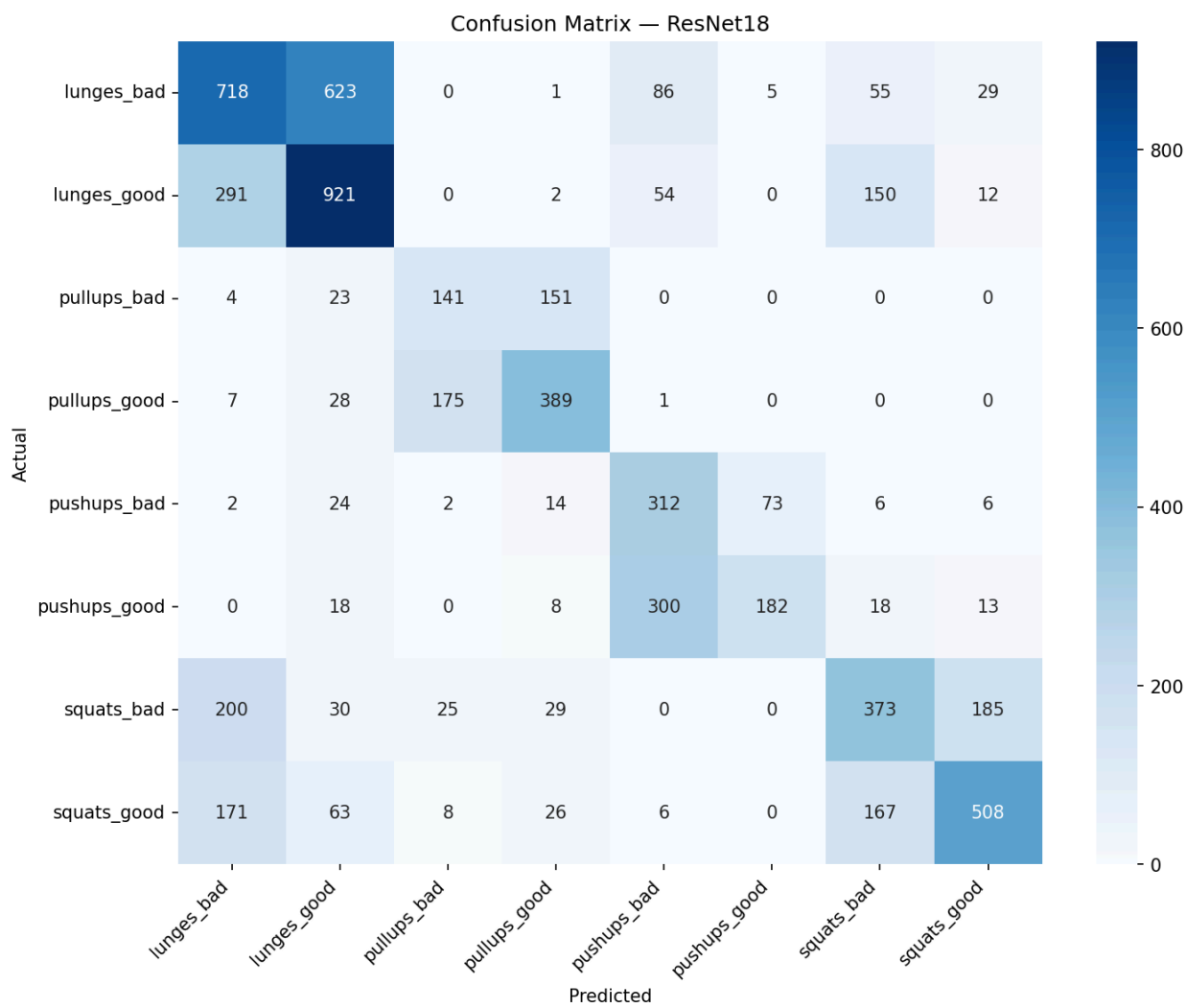
XGBoost achieved the best performance among the models, with 76% accuracy and a macro-F1 score of 0.76. Although the Transformer also achieved similar accuracy, it was through a smaller test set, which reduces the reliability of the result.

The ResNet-18 network achieved the lowest accuracy and showed severe overfitting, with training accuracy reaching nearly 96% but validation accuracy plateauing at less than 55%. This is shown in the training/validation loss graphs on the next page. There is also no clear trend in the confusion matrix, which further solidifies that the ResNet model did not learn any meaningful information.

Loss and Accuracy Graphs for ResNet Training/Validation



Feature Confusion Matrix for ResNet



Per-Class F1 Score Heatmap



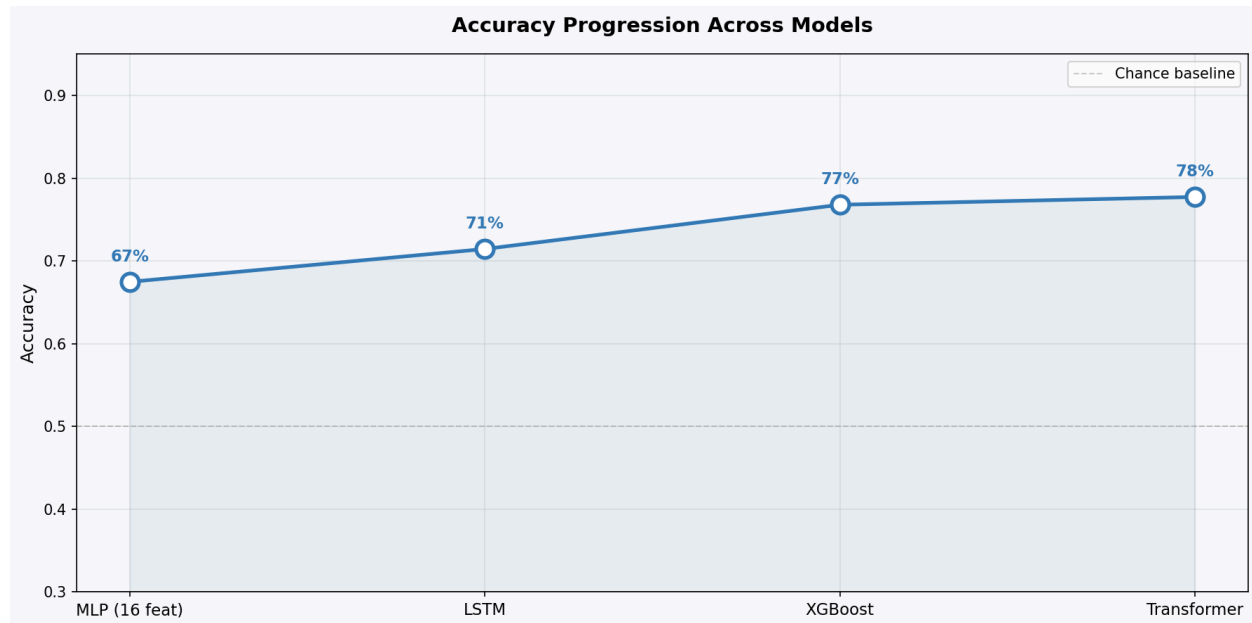
As shown above, the *pullups_bad* class consistently performed the worst across all models (with the Transformer performing the worst with a score of 0.28), most likely due to data limitations since pullups were least represented in our dataset (only 105 videos as opposed to lunges with 130 videos). Interestingly, although pushups were also significantly underrepresented in our dataset, both the XGBoost and Transformer models had the highest F1 scores for *pushups_good*, 0.92 and 0.82, respectively. Other notable, well-learned classes were the *lunges_good* and *lunges_bad* classes, which also consistently outperformed other classes across all models.

UCF-101 + YouTube (Baseline and ResNet-18 are excluded)

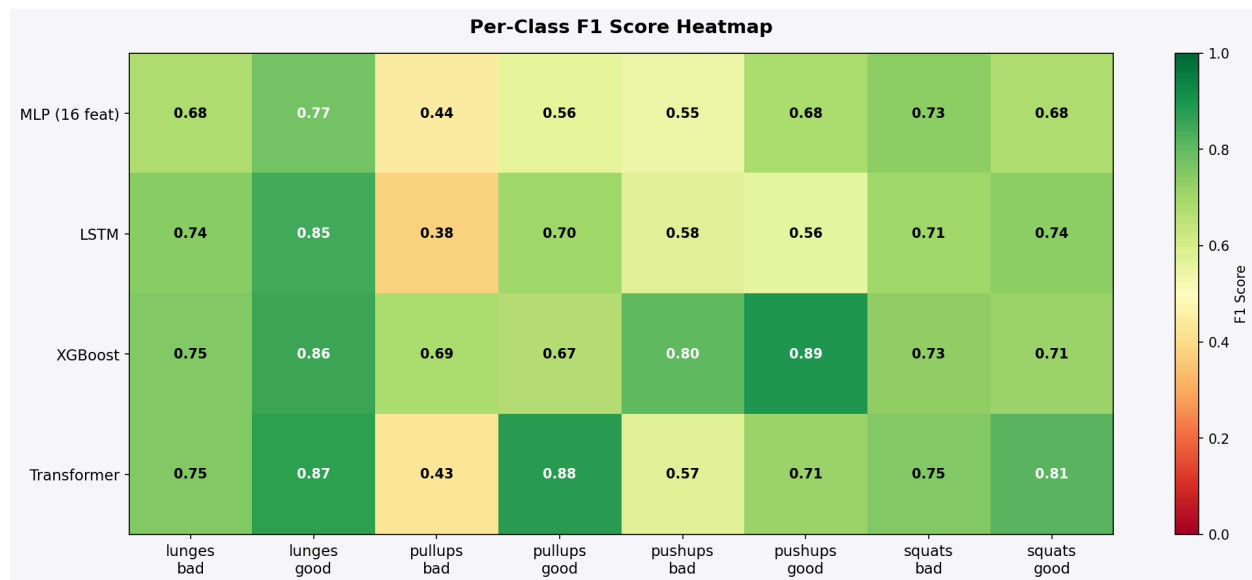
Model	Accuracy (%)	Macro-F1 Score
MLP (16 feats)	67	0.64
LSTM	71	0.66
XGBoost	77	0.76
Transformer	78	0.72

As can be seen from above, although there is not a significant improvement in accuracy, each model improved by around 2-3%. More importantly, the per-class F1 score performance (shown below) had significant improvements to the *pullups_bad* class.

Accuracy Progression Across all Models

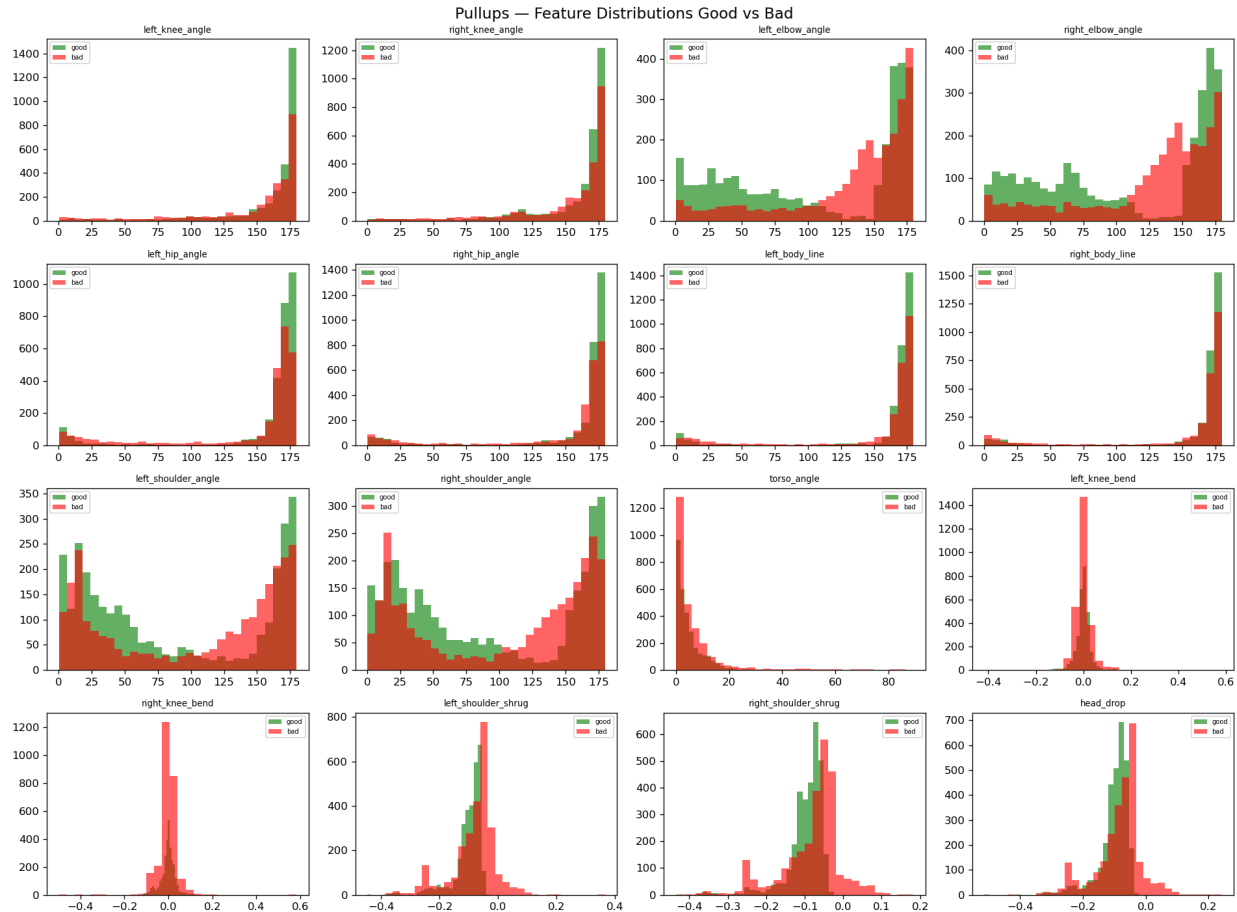


Per-Class F1 Score Heatmap



With the addition of the YouTube videos, *pullups_bad* improved for both the Transformer (from 0.28 to 0.43) and the MLP (0.36 to 0.44). Additionally, the overall performance for each class generally improved for each model.

We also conducted further analysis of the *pullups_bad* class. This class generally received the worst performance across every model, due to its lack of representation in the dataset. To understand why the models were failing to learn, we created a plot of the features and how they were distributed for both “good” and “bad” labels. As can be seen below, there is a significant overlap between both labels for each feature, which provides an explanation as to why the models are unable to distinguish meaningful differences.



c. Ablation Study

We conducted several ablation experiments to improve our models and to understand the significance of their components:

1. Feature Engineering for MLP

- Expanding from 4 features (elbow and knee angles only) to 16 (adding hip angles, body line, torso angle, shoulder shrug, knee bend, and head drop) improved the MLP model’s accuracy from 45% to 64%, achieving a large improvement.

2. Labelling With Respect to Orientation

- Implementing separate form rules for videos with recordings facing the camera versus facing off to the side significantly improved the quality of the generated labels. Before fixing this, none of the frames produced any “good” squat labels because the torso angle

was incorrectly computed for each orientation. After this change, each exercise had a relatively even distribution of “good” and “bad” labels.

3. Model Complexity versus Dataset Size

- ResNet-18, with 11.7 million parameters, achieved lower accuracy and overall results compared to XGBoost, with significantly fewer parameters. This showed that model capacity has an adverse effect when presented with less training data (441 videos from the UCF-101 dataset). MLP and XGBoost performed better because they utilise features instead of raw pixels to train on.

4. Increasing Amount of Data

- Initially, we only used UCF-101 videos to train and test our models. However, since pullups were universally underrepresented, we decided to add YouTube videos for each exercise as well.
- UCF-101 only:
 - 19006 train frames; 6682 test frames
- UCF-101 + YouTube:
 - 29548 train frames; 6682 test frames
- Overall, each model improved by around 2-3%, which isn't a significant improvement, but demonstrates the need for better data across all classes.
- Note: since adding the YouTube videos significantly increased our data size, we couldn't save each frame individually. This affected our ResNet model, which relied on raw frames for training. Given this, we ultimately decided to remove ResNet from the UCF-101 + YouTube comparison, since it didn't perform well anyway.

d. Future Directions

- We will consider expanding the size of our dataset, particularly for the pullups class, which is underrepresented in our current dataset.
- Additionally, we would label a subset of the data and train on that subset. Although our current classification method of automatic angle calculations provides an accurate base of data to train on, there are certain limitations which may only be fully captured by manual labelling.

5. Supplementary Material

https://github.com/alliedinh/Fitness_Trainer

https://www.canva.com/design/DAHDzs_iO54/9hMz81Y1s9flurRkWQ5QSg/watch?utm_content=DAHDzs_iO54&utm_campaign=designshare&utm_medium=link2&utm_source=uniquelinks&utlId=hc3ad8a2075

References

Soomro, Khurram, et al. *UCF101: A Dataset of 101 Human Actions Classes From Videos in The Wild*, 3 Dec. 2012, arxiv.org/pdf/1212.0402.

Negi, Shashank, et al. *Real-Time Human Pose Estimation: A MediaPipe and Python Approach for 3D Detection and Classification* | *IEEE Conference Publication* | *IEEE Xplore*, ieeexplore.ieee.org/document/10390506.

Chen, Tianqi, and Carlos Guestrin. *XGBoost: A Scalable Tree Boosting System* Tianqi Chen *University of Washington*, 10 June 2016, www.kdd.org/kdd2016/papers/files/rfp0697-chenAemb.pdf.